Dynamic Analysis & Monitoring Tools

Sandboxes, System Monitoring, Registry Analysis, and Network Forensics

Dr. Manu VT

NFSU Bhopal

January 22, 2026

Agenda

What is Dynamic Analysis?

Definition

Dynamic analysis (Behavioral Analysis) involves running malware in a controlled, isolated environment to observe its behavior, interaction with the system, and network communications.

Why use Dynamic Analysis?

- **Obfuscation:** Static analysis is often defeated by packing or encryption. Dynamic analysis bypasses this because the malware must unpack itself to run.
- **Intent:** It reveals *what* the malware actually does (e.g., stealing passwords, encrypting files) rather than just what the code looks like.

Automated Sandboxing

Sandboxes are security mechanisms for running untrusted programs in a safe, isolated environment. They automate the process of loading malware and recording its activity.

Common Tools:

- **Cuckoo Sandbox:** Open-source, highly customizable automated analysis system.
- **Any.Run:** Interactive web-based sandbox allowing user input.
- **Joe Sandbox:** Deep analysis with extensive reporting.

Pros & Cons:

+ **Speed:** Rapidly analyze mass amounts of samples.
+ **Safety:** No risk to the host machine.
- **Evasion:** Sophisticated malware checks for virtual environments (Anti-VM) and may refuse to run.

Limitations of Sandbox Analysis

- Sandboxes typically execute malware **without command-line options**.
  - Malware requiring specific options may not trigger malicious behavior.
- Malware that waits for **command-and-control (C2) communication** may not activate.
  - Backdoors are not launched if no C2 packet is received.
- **Time-based evasion** can prevent detection.
  - Malware may sleep for extended periods before acting.
  - Although sleep functions are often hooked, alternative delay methods exist.
- Sandboxes may **fail to capture all events**.
  - Analysis duration may be insufficient.

Additional Sandbox Drawbacks

- Malware may detect execution in a **virtual machine**.
  - Behavior may change or execution may stop entirely.
- Required **registry keys or files** may be missing.
  - These may contain configuration data, commands, or encryption keys.
- **DLL-based malware** may not execute properly.
  - Exported functions are not always invoked.
- **Operating system mismatch** can affect behavior.
  - Malware may fail on one OS version but succeed on another.
- Sandboxes provide **limited semantic understanding**.
  - High-level conclusions about malware purpose require analyst judgment.

What is Cuckoo Sandbox?

Definition

Cuckoo Sandbox is an advanced, open-source **automated malware analysis system**. It allows analysts to execute suspicious files in a safe, isolated environment (a "sandbox") and automatically record their behavior.

The Core Problem it Solves:

- Manual analysis is slow and risky.
- Static analysis (reading code) can be defeated by obfuscation.
- Cuckoo answers the question: *"What does this file actually DO?"*

What Can It Analyze?

Cuckoo is versatile and supports various analysis targets:

Target Types:

- **Windows Binaries:** .exe, .dll, .com
- **Documents:** PDF, MS Office (Word, Excel with macros)
- **Scripts:** PowerShell, VBScript, Python
- **Web Content:** URLs, HTML (Drive-by downloads)

Output Data:

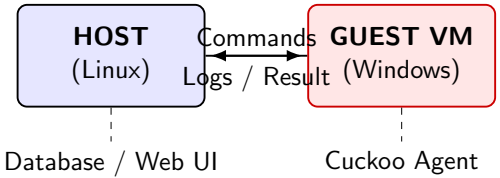
- Native API calls traces.
- Network traffic (PCAP).
- Screenshots of the desktop.
- Memory dumps of the analyzed process.

How It Works: The Architecture

Cuckoo relies on a **Host-Guest** architecture to ensure safety.

1. **The Host (The Brain):**
   - Usually Linux (Ubuntu/Debian).
   - Manages the database, web interface, and scheduling.
   - Never runs the malware directly.

2. **The Guest (The Victim):**
   - Virtual Machines (Windows 7/10, Android).
   - Isolated "Host-Only" network.
   - Reverts to a clean snapshot after every run.



System Architecture

The architecture relies on a **Host-Guest** model. They communicate over a strictly controlled virtual network.

The Host (Orchestrator)

- **Cuckoo Core & Scheduler:**
  Manages tasks, reverts VMs.
- **Database:**
  MongoDB / PostgreSQL storage.
- **Result Server:**
  Listens for logs streaming from Guest.
- **Auxiliary Tools:**
  TCPDump, Volatility.

Virtual Network

The Guest (Victim VM)

- **Virtual Machine:**
  Windows / Linux / Android.
- **Agent.py:**
  XML-RPC server listening for commands.
- **Malware Sample:**
  The payload being analyzed.
- **API Hooks:**
  Injected into the process to trace calls.

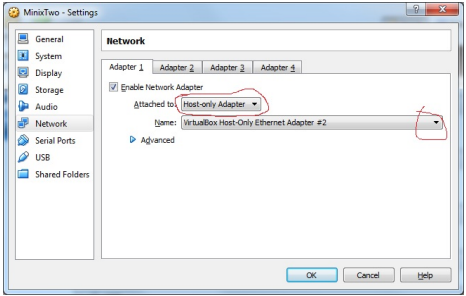
Role of the Agent

The Agent (agent.py) runs inside the Guest VM (the isolated environment).

- It functions as an XML-RPC server.
  - RPC (Remote Procedure Call) is a communication paradigm that allows a program to execute a procedure (function or method) on a remote system as if it were a local call.
- It listens for commands sent by the Host machine. These commands instruct the VM to perform actions like uploading the malware sample, executing it, taking screenshots, or stopping the analysis.

This architecture allows the Host to control the Guest VM remotely without needing complex network setups like SSH or RDP inside the victim machine.

What is Host-Only Networking?



- A VirtualBox networking mode for isolated communication
- Allows communication between:
  - Host machine and virtual machines
  - Virtual machines on the same host-only network
- No access to:
  - Internet
  - External physical LAN

How Host-Only Networking Works

- VirtualBox creates a virtual network adapter on the host
- Virtual machines attach to this adapter
- Acts as a private Ethernet switch
- Optional DHCP server assigns IP addresses

Conceptual Topology:

$$\text{Host OS} \leftrightarrow \text{Host-Only Adapter} \leftrightarrow \text{VMs}$$

Comparison with Other Network Modes

Mode	VM–Host	VM–VM	Internet	LAN
NAT	Limited	No	Yes	No
Bridged	Yes	Yes	Yes	Yes
Host-Only	Yes	Yes	No	No
Internal	No	Yes	No	No

Configuration Steps

1. Create a Host-Only Adapter
   - VirtualBox Tools → Network
   - Host-only Networks tab
2. Assign Adapter to VM
   - VM Settings → Network
   - Attached to: Host-only Adapter
3. (Optional) Enable DHCP or configure static IPs

Advantages

- Strong isolation from external networks
- Predictable and stable IP configuration
- Easy to configure and manage
- Ideal for testing and development

Limitations

- No internet access by default
- Not accessible from other physical machines
- Requires additional adapters for routing

The Analysis Workflow

What happens when you click "Submit"?

1. **Submission:** The user submits a sample (file or URL) to the Host.
2. **Scheduling:** The Host looks for an available Guest VM.
3. **Preparation:** The Guest VM is reverted to a clean "Snapshot" to ensure no previous contamination.
4. **Injection:** The Host pushes the malware + the **Cuckoo Agent** into the Guest.
5. **Execution:** The Agent executes the malware.
6. **Monitoring:** The Agent hooks APIs inside the VM; The Host sniffs network traffic outside the VM.
7. **Reporting:** Data is sent back to the Host, processed, and a report is generated.

How Cuckoo "Sees" Behavior

1. API Hooking (Inside the Guest)

Cuckoo injects a monitoring DLL into the malware process. This technique is called **API Hooking**.

- It intercepts calls to the Windows OS (e.g., `CreateFile`, `RegSetValue`, `InternetOpen`).
- It logs the arguments (e.g., *"Malware tried to open C:\Windows\System32\cmd.exe"*).

2. Network Sniffing (Outside the Guest)

The Host runs `tcpdump` on the virtual network interface.

- Captures all traffic leaving the VM.
- Can decrypt SSL/TLS if Man-in-the-Middle (MitM) is configured.

API Hooking and Monitoring in Cuckoo Sandbox

- **Injection:** Cuckoo injects a monitoring DLL into the malware process at execution time.
- **Hooking:** The injected DLL modifies the entry points of standard Windows API functions (such as `CreateFileW`, `InternetConnectA`, or `RegSetValueEx`). It inserts a trampoline or detour instruction to redirect execution flow.
- **Interception:** API calls are redirected to monitoring code.
- **Logging:** Function names, arguments, and return values are recorded.
- **Pass-through:** Execution resumes at the original API function.

Monitoring data is streamed in real time to the host-side Result Server.

Trampoline or Detour instruction

- Concept refers to a low-level technique called Inline API Hooking, which is how Cuckoo intercepts the malware's communication with the operating system.
  1. The Normal State (Before Hooking)
     - Imagine the Windows function **CreateFileW** (used to open files) as a set of instructions in memory. When a program calls it, the CPU goes to that address and executes the instructions linearly:

```
Address | Instruction
--------------------
0x1000  | MOV ... (Standard setup instruction)  <-- Entry Point
0x1005  | PUSH ...
0x100A  | CALL ... (Do the actual work)
```

  2. The Detour (The Injection)
     - To "hook" this function, Cuckoo overwrites the first few bytes of **CreateFileW** with a Jump instruction (a "Detour"). This instruction forces the CPU to jump to Cuckoo's own monitoring code instead of running the original function immediately.

```
Address | Instruction
--------------------
0x1000  | JMP 0x9999 (JUMP to Cuckoo's DLL)     <-- The "Detour"
0x1005  | PUSH ...
0x100A  | CALL ...
```

Trampoline or Detour instruction (Contd...)

- 3. The Trampoline (The Bridge) :
  If Cuckoo just overwrote the first instruction, the original function would be broken because that first instruction (MOV ...) is gone.
  To fix this, Cuckoo creates a Trampoline. The trampoline is a small, dynamically generated block of code elsewhere in memory that does two things:
  1. It contains the original instructions that were overwritten (stolen) to make room for the Detour.
  2. It contains a jump back to the original function, landing right after the Detour.

```
Address | Instruction
--------------------
0x8000  | MOV ... (The original stolen instruction)
0x8005  | JMP 0x1005 (Jump back to original function, skipping the detour)
```

Summary of the Flow

When the malware tries to open a file:

1. **API Invocation:** The malware calls `CreateFileW`.
2. **Detour Execution:** The CPU encounters the injected `JMP` instruction at the function entry point and redirects execution to Cuckoo's logging routine.
3. **Logging:** Cuckoo records the event, noting that the malware is attempting to open `secret.txt`.
4. **Trampoline Invocation:** Cuckoo invokes the trampoline function associated with the hooked API.
5. **Original Code Execution:** The trampoline executes the saved original instruction (e.g., a `MOV`) and then jumps back to the original `CreateFileW` code, past the hook.
6. **Result:** The file is opened successfully, and the malware continues execution without awareness of the interception.

The Lab Environment

To perform manual dynamic analysis, a specific architecture is required.

Safety First

Never run malware on your host operating system. Always use a Virtual Machine (VM).

1. **Virtualization:** Use VMWare or VirtualBox.
2. **Network Isolation:** Configure network adapters to **Host-Only**. This prevents malware from reaching the real internet while allowing communication with a simulated network.
3. **Snapshots:** Take a "Clean State" snapshot **before** infection. Revert to this snapshot immediately after analysis to erase all traces of the malware.
4. **Baselining:** Monitor the system *before* executing the malware to understand normal background noise.

Process Monitor (ProcMon)

Tool: Sysinternals Suite
ProcMon is an advanced monitoring tool for Windows that displays real-time file system, Registry, and process/thread activity.

Key Capabilities

- **File System:** Detects when malware drops files (payloads) or deletes system files.
- **Registry:** Tracks keys used for persistence (e.g., `HKCU\...\Run`).
- **Filtering:** ProcMon captures millions of events. You **must** use filters (e.g., `Process Name is malware.exe`) to find relevant data.

Analysis Tip: Look for "WriteFile" operations to see if the malware is extracting a secondary stage downloader.

Process Explorer (ProcExp)

Tool: Sysinternals Suite

Often described as "Task Manager on steroids." It provides unique insight into the hierarchy of running processes.

- **Process Tree:** Shows parent-child relationships.
  - *Suspicious:* `AdobeReader.exe` spawning `cmd.exe`.
- **Verify Signatures:** Automatically checks digital signatures.
  - *Suspicious:* A process named `svchost.exe` that is NOT signed by Microsoft.
- **Strings in Memory:** Allows you to view the strings of a running process. This often reveals unpacked URLs or IP addresses that were hidden during static analysis.
- **Handles/DLLs:** Shows which files or libraries the malware has hijacked.

RegShot: Registry Comparison

Malware almost always modifies the registry to ensure it starts automatically when the computer reboots (Persistence).

The "Diffing" Workflow

1. **1st Shot:** Launch RegShot on a clean machine and take a snapshot.
2. **Infection:** Execute the malware sample and wait for activity to settle.
3. **2nd Shot:** Take a second snapshot.
4. **Compare:** Click "Compare". RegShot produces a text log showing **only** the changes.

What to look for: Changes to firewall settings, browser homepages, or keys added to `CurrentVersion\Run`.

Faking a Network (Network Simulation)

The Problem

If you disconnect the VM from the internet, malware trying to reach a Command & Control (C2) server may fail and stop running (terminate).

The Solution: INetSim / ApateDNS

We simulate the internet so the malware *thinks* it is online.

- **DNS Spoofing:** Resolves **all** domain requests (e.g., `evil.com`) to the analysis machine's local IP.
- **Service Simulation:** Listens on common ports (80, 443, 21, 25) and returns standard "OK" responses.
- **Result:** The malware sends its requests to us, allowing us to capture the traffic without letting it reach the real internet.

Using Wireshark for Packet Analysis

Wireshark captures network packets flowing in and out of the monitored interface.

Key Indicators:

- **DNS Queries:** Reveals the domains the malware is trying to contact (DGA domains).
- **HTTP Streams:** Shows the content of web requests (User-Agents, POST data).
- **TCP Handshakes:** Failed connections (SYN packets) indicate scanning behavior.

Essential Features:

- **Follow TCP Stream:** Reassembles packets into a readable conversation. Useful for seeing stolen data being exfiltrated.
- **Filters:**
  - `http.request.method == "POST"`
  - `ip.addr == x.x.x.x`
  - `dns`

Summary: The Analysis Cycle

Dynamic analysis requires an iterative approach using complementary tools:

1. **Setup:** Prepare the Sandbox/VM. Start **Wireshark** and **ProcMon**. Take **RegShot** (Shot 1).
2. **Run:** Execute the malware. Interact if necessary.
3. **Observe:** Watch **Process Explorer** for new processes and handles. Watch network simulation logs.
4. **Conclude:** Take **RegShot** (Shot 2). Stop captures. Analyze the logs to identify Indicators of Compromise (IOCs).

Thank You

Questions?